

Pyontrust

Integration & Workflow Tutorial

Embedded Test & Measurement Platform
v2026.3.0 — CalVer

What is Pyontrust?

A **pure-Python** platform for automated hardware testing:

Measure

- Power / current profiling
- Thermal camera imaging
- CAN bus diagnostics
- Analog waveform capture

Automate

- FlowLab visual block graph
- Lab bench orchestration
- Pass / fail limit checking
- PDF / CSV / HTML reports

Key idea — Every instrument, recorder, and analysis module plugs into a common **EventBus** so you can compose arbitrary test flows.

Architecture Overview

```
+-----+
|           Flask Gateway  :5200           |
| blueprints: shell, diag, flowlab, thermal ... |
+-----+
| Analysis | Services | Recorders |
| - power  | - event_bus | - csv    |
| - thermal | - config  | - plot   |
| - limits | - hw_disc  | - pdf    |
+-----+
|           HAL  (Hardware Abstraction)      |
| AD3 / DMM / Thermal / PCAN / GPIO / Relay  |
+-----+
```

Every layer is importable on its own — no gateway required for scripting.

Installation

1. Clone & install (editable)

```
git clone https://github.com/your-org/pyontrust.git
cd pyontrust
pip install -e ".[dev]"
```

2. Hardware drivers (optional)

```
# Digilent WaveForms runtime -> AD3
# PEAK PCAN-Basic           -> CAN bus
# libusb + pyusb            -> Seek Thermal
python scripts/discover_hardware.py # verify
```

3. Start the gateway

```
python -m pyontrust.gateway.app # http://localhost:5200
```

Tip — Run `discover_hardware.py` first. It prints a table of every detected instrument so you know what is available before writing tests.

Project Structure

```
pyontrust/  
+-- src/pyontrust/  
|   +-- core/           # models, event bus, config  
|   +-- hal/            # hardware abstraction drivers  
|   +-- instruments/    # high-level instrument wrappers  
|   +-- recorders/      # CSV, plot, PDF recorders  
|   +-- analysis/       # power, thermal, limits  
|   +-- services/       # hardware discovery, utilities  
|   +-- gateway/        # Flask app + blueprints + web UI  
+-- tests/              # 665+ unit tests (pytest)  
+-- profiles/           # JSON test profiles  
+-- limits/             # JSON pass/fail limits  
+-- benches/           # board definition files  
+-- scripts/            # CLI helpers
```

Each folder maps 1-to-1 with an architecture layer.

Core Models

All data flows through **dataclasses**:

```
@dataclass
class Sample:
    timestamp: float
    value: float
    unit: str = "A"

@dataclass
class PowerTrace:
    samples: list[Sample]
    metadata: dict
```

Domain Objects

- `Sample` — single reading
- `PowerTrace` — ordered samples
- `ThermalFrame` — pixel matrix
- `LimitSpec` — pass/fail rule

Why dataclasses?

- Immutable-friendly
- Free `__repr__` / `__eq__`
- Easy JSON serialization
- Type-checker friendly

HAL — Hardware Abstraction

Every driver implements a thin **protocol**:

```
class PowerMeterProtocol(Protocol):  
    def open(self) -> None: ...  
    def read_current(self) -> float: ...  
    def close(self) -> None: ...
```

Concrete drivers:

AD3 Power

Analog Discovery 3
via DWF C library

Seek Thermal

Pure-Python USB
libseek driver

PCAN

CAN bus diag
via PCAN-Basic

Swap rule — Any driver can be replaced with a simulated stub by setting `simulated: true` in the bench file. Tests run everywhere.

Tutorial 1 — Power Measurement

```
from pyontrust.instruments import AD3PowerMeter

meter = AD3PowerMeter(shunt_ohm=0.1)
meter.open()

trace = meter.record(duration_s=5.0, rate_hz=1000)
avg    = sum(s.value for s in trace.samples) / len(trace.samples)
print(f"Average current: {avg*1e6:.1f} uA")

meter.close()
```

What happens

1. `open()` — loads DWF, configures ADC
2. `record()` — streams samples
3. Returns `PowerTrace` dataclass

Typical use-cases

- Sleep-current profiling
- TX burst energy calc
- Battery-life estimation

Tutorial 2 — Thermal Camera

```
from pyontrust.hal.seek_thermal import SeekThermalDriver

cam = SeekThermalDriver()
cam.open()
frame = cam.grab_frame()          # ThermalFrame
print(f"Hot-spot: {frame.max_temp:.1f} C")
cam.close()
```

Four measurement modes

Continuous — stream N frames

Soak — record until temperature stabilises

Delta — measure temperature change

Gradient — spatial gradient across zone

The `thermal_measurement` blueprint exposes all four modes via REST and the dashboard auto-plots live data.

Tutorial 3 — Lab Bench Definition

Define your bench in JSON:

```
{
  "name": "nrf9160dk",
  "instruments": {
    "power": {
      "type": "ad3",
      "shunt_ohm": 0.1,
      "simulated": false
    },
    "thermal": {
      "type": "seek_compact_pro",
      "simulated": true
    }
  }
}
```

```
from pyontrust.core import LabBench
bench = LabBench.from_file("benches/nrf9160dk.json")
bench.open_all()          # opens every instrument
```

Simulated mode lets your CI run the same profiles without hardware.

Tutorial 4 — Web Dashboard

Shell (landing page)

- Links to every blueprint
- System status at a glance
- Quick-launch buttons

Thermal Dashboard

- 4 tabs: Measure, Live, Results, Reports
- Real-time frame display
- Zone-of-interest selection

FlowLab

- Drag & drop block editor
- 90+ blocks in registry
- One-click code generation

CAN Diagnostics

- DTC scanner
- Live frame monitor
- UDS service calls

All dashboards are single-page apps served by the Flask gateway.

Tutorial 5 — FlowLab Visual Editor

FlowLab is a **node-graph** editor for test flows:

```
{
  "blocks": [
    { "id": "b1", "type": "open_instrument",
      "params": { "bench": "nrf9160dk" } },
    { "id": "b2", "type": "power_record",
      "params": { "duration_s": 5, "rate_hz": 1000 } },
    { "id": "b3", "type": "check_limits",
      "params": { "limit_file": "limits/sleep_current.json" } },
    { "id": "b4", "type": "plot_trace",
      "params": { "title": "Sleep Current" } }
  ],
  "edges": [
    ["b1", "b2"], ["b2", "b3"], ["b2", "b4"]
  ]
}
```

Each block is a Python function — the engine topologically sorts the graph and runs blocks in dependency order.

FlowLab — Block Categories

Instruments

`open_instrument`

`power_record`

`dmm_read`

`thermal_grab`

Analysis

`check_limits`

`calc_energy`

`thermal_delta`

`statistics`

Output

`plot_trace`

`csv_export`

`pdf_report`

`mqtt_publish`

Flow Control

`if_else`

`loop`

`delay`

`parallel`

CAN Bus

`can_open`

`can_send`

`can_scan_dtc`

`can_uds`

Thermal

`thermal_measure`

`thermal_soak`

`thermal_delta`

`thermal_gradient`

90+ blocks — all discoverable via `/flowlab/blocks` API.

Tutorial 6 — Writing a Custom Block

```
# In flowlab_engine.py – register a new block

def _blk_voltage_divider(inputs, params, ctx):
    """Calculate divided voltage from two resistor values."""
    v_in = inputs["voltage"]
    r1    = params.get("r1", 10_000)
    r2    = params.get("r2", 10_000)
    v_out = v_in * r2 / (r1 + r2)
    return {"voltage_out": round(v_out, 6)}

BLOCK_REGISTRY["voltage_divider"] = {
    "fn":          _blk_voltage_divider,
    "label":       "Voltage Divider",
    "category":    "analysis",
    "inputs":      ["voltage"],
    "outputs":     ["voltage_out"],
    "params":      {"r1": 10000, "r2": 10000},
    "description": "Resistive voltage divider calculation",
}
```

Convention — prefix with `_blk_`, return a dict of named outputs, register in `BLOCK_REGISTRY`.

Tutorial 7 — Event Bus

The `EventBus` decouples producers from consumers:

```
from pyontrust.core.event_bus import EventBus

bus = EventBus()

# Subscribe
bus.subscribe("sample", lambda evt: print(evt))

# Publish
bus.publish("sample", {"value": 3.3, "unit": "V"})
```

Built-in topics

- `sample` — new measurement
- `frame` — thermal frame
- `limit_result` — pass / fail
- `profile_done` — run finished

Architecture benefit

- Instruments publish data
- Recorders subscribe
- Zero coupling between layers
- Easy to add new consumers

Tutorial 8 — Writing & Running Tests

```
# tests/power_framework_tests/test_limits.py
import pytest
from pyontrust.analysis.limits import check_limits

def test_sleep_current_pass():
    trace = make_trace(avg_uA=4.2)
    result = check_limits(trace, "limits/sleep_current.json")
    assert result.passed is True

def test_sleep_current_fail():
    trace = make_trace(avg_uA=150.0)
    result = check_limits(trace, "limits/sleep_current.json")
    assert result.passed is False
```

```
pytest tests/ -v --tb=short      # 665+ tests, 13 skipped
```

Skipped tests are hardware-dependent — they run only when the real instrument is connected. CI always stays green.

Tutorial 9 — Adding a New Driver

```
# src/pyontrust/hal/my_sensor.py
from pyontrust.hal.protocols import SensorProtocol

class MySensorDriver(SensorProtocol):
    def open(self):
        self._dev = connect_usb(vid=0x1234, pid=0x5678)

    def read(self) -> float:
        raw = self._dev.bulk_read(64)
        return _parse_temperature(raw)

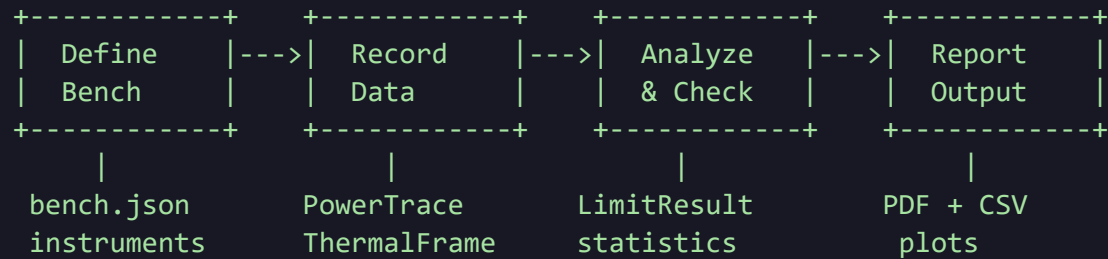
    def close(self):
        self._dev.close()
```

Checklist for a new driver

1. Implement the matching `Protocol`
2. Add a simulated stub for CI
3. Register in `hardware_discovery.py`
4. Write tests (real + simulated)
5. Add a FlowLab block if useful

Following this pattern guarantees the driver works with every existing recorder, analysis module, and FlowLab block.

End-to-End Workflow



Script mode

```
bench = LabBench.from_file(...)
bench.open_all()
trace = bench["power"].record(5)
result = check_limits(trace, ...)
generate_report(result)
```

FlowLab mode

1. Open dashboard
2. Drag blocks onto canvas
3. Connect edges
4. Click **Run**
5. Download PDF report

Hardware Compatibility Matrix

Instrument	Interface	Driver	Simulated
Analog Discovery 3	USB	<code>ad3_dwf</code>	Yes
Seek Compact Pro	USB	<code>libseek</code>	Yes
PCAN-USB	USB/CAN	<code>pcan_basic</code>	Yes
Keithley DMM	SCPI	<code>scpi_dmm</code>	Yes
GPIO Expander	I2C	<code>gpio_i2c</code>	Yes
Relay Board	Serial	<code>relay_serial</code>	Yes

Every driver has a `simulated: true` option. CI test runs never require physical hardware.

Quick Reference

CLI Commands

```
# Start gateway
python -m pyontrust.gateway.app

# Run all tests
pytest tests/ -v

# Discover hardware
python scripts/discover_hardware.py

# Run a profile
python scripts/run_profile_new.py \
  profiles/sleep_current.json
```

REST API Endpoints

GET	/	shell
GET	/diag/status	diagnostics
GET	/flowlab/	editor
POST	/flowlab/run	execute graph
GET	/flowlab/blocks	block list
GET	/thermal/	dashboard
POST	/thermal/measure	start meas
GET	/can/	CAN diag
POST	/bench/run	run profile

Summary

What you learned

1. Project structure & layers
2. Core dataclass models
3. HAL protocol pattern
4. Power & thermal measurement
5. Lab bench orchestration
6. FlowLab visual editor
7. Event bus architecture
8. Writing tests & drivers

Key takeaways

- **Protocol-based HAL** — swap drivers freely
- **Simulated stubs** — CI runs without hardware
- **EventBus** — zero coupling between layers
- **FlowLab** — no-code test composition
- **CalVer** — `v2026.3.0`
- **665+ tests** — full coverage

Start with `discover_hardware.py`, build from there.

Thank You

Questions? Open an issue or check the docs.

Pyontrust v2026.3.0 | Python 3.10+ | Flask 3.x | 665+ Tests